SP-108 Green ASM

Project Development Document

CS 4850 - Section 01 – Spring 2026

03/22/2026

Professor Perry

Alex Tidwell Team Leader Documentation + Developer (as needed)	Katherine Smith Documentation + Developer	Sumaya Tamenne Documentation	Phillip Eiser Developer	Brianna Lewis Developer

Table of Contents

Table of Contents 1

1. Implementation Detail Narrative 2

1.0 System Integration & Workflow Overview 2

1.0.0 The Integrated Process: 2

1.0.1 Operational Workflow: 2

1.0.2 Global Design Decisions: 2

1.1 Target Environment Component 3

1.1.0 Conceptual Overview: 3

1.1.1 Development Process: 3

1.2 Scanning & Analysis Component 4

1.2.0 Conceptual Overview: 4

1.2.1 Development Process: 4

2. Database and External Connections 4

2.1 Connection Overview: 5

2.2 Configuration Details: 5

2.3 Integration Logic: 5

3. Project Setup and Environment Configuration 5

3.1 Prerequisites 5

3.2 Installation Steps 5

3.3 Verification 6

4. Development Conclusion or Summary 6

4.1 Current State Overview 6

4.2 Known limitations 6

4.2.1 Hardcoded Credentials 7

4.2.2 Discord Message Splitting 7

4.2.3 DNS Port 53 Issues 7

4.2.4 Python Virtual Environment 7

1. Implementation Detail Narrative

1.0 System Integration & Workflow Overview

1.0.0 The Integrated Process:

This project implements an Attack Surface Management (ASM) system that continuously scans a target environment, identifies vulnerabilities, and delivers security notifications to users.

There are a handful of tools that work together for this system to work end-to-end and simulate a real-world ASM workflow. An automation layer using cron and a Bash script initiates the scans, and OWASP ZAP analyzes the target systems' attack surface. Docker containers are used to create the target environment, which simulates a real infrastructure with multiple services. To transform raw scanner data into more user-friendly summaries, we have a processing layer implemented in Python using the Ollama API. Lastly, we use Discord Webhooks to deliver alerts to the user. All of these layers and tools communicate in an ordered manner to represent a simplified yet realistic ASM pipeline, where discovery, analysis, and reporting are automated.

1.0.1 Operational Workflow:

An example execution of the ASM would follow this structure:

First, the cron job would trigger the Bash script weekly on Monday. The Bash script will launch OWASP ZAP, and OWASP will scan the target VM and any exposed information. OWASP ZAP then creates a report with all found vulnerabilities and their issues. A Python script will then process the report, identify the most recent scanned file, ingest its contents, and send the report to the Ollama API. Ollama API will generate a readable summary of the report, which is saved locally and sent to users via Discord webhook for alerts. If the summary exceeds the webhook's limit, it is split into multiple messages.

1.0.2 Global Design Decisions:

While many other tools can achieve the same outcome as the selected ones, it was important to select those we believed would benefit the overall function of the system. OWASP ZAP was used to scan the target environment and was selected for its strong documentation and beginner-friendly nature. We used Docker to create the Target environment because it allows for the simulation of a realistic attack surface, which is important for the testing phase. Ollama was used for summarization because it does not

rely on external cloud APIs, enabling greater control. The Bash and Python languages were selected for their simplicity while still allowing for automation, scheduling, and processing. Discord Webhooks were selected for alerting because they provide a simple way to deliver alerts to users without requiring a dedicated backend. Other decisions made include hard coding the API keys for simplicity while debugging. In the future, environment files and new scripts can be created for more security.

1.1 Target Environment Component

1.1.0 Conceptual Overview:

The target environment component represents the attack surface that the ASM monitors. Its focus is to simulate and represent a realistic infrastructure with many services, allowing the ASM to scan for vulnerabilities. The environment is designed to represent a real-world system, including a combination of web services and networking components.

1.1.1 Development Process:

The target environment was designed using Docker Compose. Docker allows for the creation of containers that can be configured and managed together. Using Docker made the setup easier and kept the simulation flexible to future change.

During development, there were challenges with the ports. For example, port 53 on Ubuntu needed to be cleared in order for CoreDNS to run properly. Issues like this were resolved by freeing the ports and ensuring no other services were using them. Despite these challenges, Docker still allowed for an effective and efficient setup of the target environment.

The environment includes a web server container, a hidden page, a MySQL database container, a Redis container, and a CoreDNS container. The web server container hosted the main application, and MySQL was used for backend data storage. The hidden page was added to test if the ASM could catch less obvious elements in the attack surface. The Redis container is used to introduce other services and vulnerabilities. The CoreDNS container is used to manage domain names and simulate realistic network routing. CoreDNS was selected because it gives us control over how services are exposed through ports and domain names.

During development, there were challenges with the ports. For example, port 53 on Ubuntu needed to be cleared in order for CoreDNS to run properly. Issues like this were resolved by freeing the ports and ensuring no other services were using them. Despite these challenges, Docker still enabled an effective, efficient setup of the target environment.

1.2 Scanning & Analysis Component

1.2.0 Conceptual Overview:

The Scanning & Analysis Component is responsible for detecting vulnerabilities within the target environment, processing results, and alerting users. This is the main function of the ASM, which enables continuous monitoring and reporting. This component combines functions such as vulnerability scanning, data processing, and alerting. All together, these functions reconstruct raw data from scans into readable data that users can act on.

1.2.1 Development Process:

Because this component includes many functions, the development happened in multiple stages.
First was the vulnerability scanning. A Bash script was created to automatically run OWASP ZAP. The schedule is controlled by the cron job, which, as of right now, runs scans every Monday. The script first launches OWASP ZAP and performs a scan; the results are saved in a directory, and the script is made executable using Linux commands.
Next, the Python script for data processing and summarization was created. The Python script locates the most recent scan report, reads the data, and sends it to the Ollama API for summarization. Ollama creates a more user-friendly summary of the scan reports, improving usability for users unfamiliar with raw vulnerability data. This script requires Python libraries such as requests, json, os, and glob, which require a virtual environment for proper installation and management.
Last was alerting and integration. After generating the summary, the system sends alerts using Discord webhooks. The Python script sends an HTTP request containing the summary; alerts are delivered to Discord, and the summary is saved locally. Once alerting was completed, it was important to ensure the workflow was executed as planned. The bash script triggers the scan, the Python script processes the report, Ollama creates the summary, and webhooks notify the user. If even one component fails, the entire system is impacted.

2. Database and External Connections

2.1 Connection Overview:

There is no current relational or document database used for this project. Discord is used currently as our main storage for all scan logs, where there is a channel handling all deliveries of scans. There are two primary external connections which include an

Ollama API instance and a Discord webhook which assists in the delivery of summarized scans.

2.2 Configuration Details:

The Ollama API endpoint is currently hardcoded, and in the future, it would be ideal to switch it to an environment variable. The key is just a localhost address; this is the connection string and is called to summarize the data from the current scans. Discord provides the URL for our webhook; this is the connection string and is currently hardcoded as well. In the future, it would be ideal to switch this to an environment variable.

2.3 Integration Logic:

First, a scan is run through OWASP Zap and produces an output. This scan is then sent to the Ollama API and is summarized as a readable report. Then this report is sent to the Discord by the webhook. Discord caps messages at approximately 2000 characters, so a split is initiated at 1900 characters. This splitting may break bullet points, which is a current issue we are having. The bash scripts are what tie all of these communications together.

3. Project Setup and Environment Configuration

3.1 Prerequisites

You will need to run Python 3.10 in order to use OWASP Zap. The setup environment for testing the ASM is a set of docker containers created using a docker compose file, with various different items inside them, such as vulnerable databases, websites, etc. Ollama is needed to generate the reports that are outputted to discord using discord webhooks. The entire environment is made to be ran on Ubuntu Server 24.04 LTS.

There are multiple python dependencies that are needed to run the python scripts necessary for performing the scans and outputting the reports. The dependencies include the Ollama package, a json package, os requests, and glob.

3.2 Installation Steps

1. Verify that you are using Ubuntu Server 24.04 LTS (Other versions may work but are untested)
2. Clone our repository from Github
3. Verify that you have Python 3.10 installed, if not install it
4. Make sure the port 53 is open and unused

5. Make sure all dependencies are setup properly and any Api keys needed are setup as environment variables
6. Set up docker compose files to scan (or just scan network with permission)
7. Run docker compose files
8. Set up cron job to execute bash scripts
9. Run scan to test everything is set up properly

3.3 Verification

You will know that the installation was done properly when the OWASP scan completes successfully, the report file appears in the scan folder, an ai summary of the report is generated in another file, and the ai summary is sent to the discord being used to receive reports.

4. Development Conclusion or Summary

4.1 Current State Overview

The current state of the build is the basic functionality for the project to work. There is a bash script which initiates the scan from OWASP Zap. This report is then sent through an Ollama API to produce a summary of what was scanned, as well as any known vulnerabilities. That summary report is then sent to Discord through a webhook and can be read by the team. The target environment is also set up and is being used to test our scans.
There are currently multiple parts that are being improved upon. The team is planning to test and add more tools. The first tool being Nmap, which would enumerate the target environment, giving the report a better overview of the attack surface. There are still some other tools that we are researching, to see if they could provide extra insight into vulnerabilities in our target. Our team is also planning to add some more vulnerabilities to our target environment, either through WordPress, Redis, or the MySQL database. This would allow us to test the ASM system to its full abilities.

4.2 Known limitations

4.2.1 Hardcoded Credentials

The Ollama API key and Discord webhook are currently hardcoded, which poses a security risk since anyone with the source code can view sensitive information and URLs. This should be changed to environment variables in the future.

4.2.2 Discord Message Splitting

Discord has a known limitation, which is that messages are limited to approximately 2000 characters. Therefore, scan reports must be split once the length has reached 1900 characters. This, however, can cause issues with bullet points and the formatting of the reports.

4.2.3 DNS Port 53 Issues

There is a known issue with the use of CoreDNS on the target environment. Port 53 is used by Ubuntu's DNS systems, meaning that this port must be freed before CoreDNS can work to properly map the domains for the target. This, however, may not be an issue unless the user is planning to set up their own test environment on an Ubuntu run system.

4.2.4 Python Virtual Environment

A Python virtual environment is needed for setup due to the various dependencies and packages. This adds additional steps and could cause issues if not set up correctly.